1. Which of the following is a compound data type in Python?

(A) str
(B) list
(C) dict
(D) All of the above

2. What built-in function returns the total number of items in a sequence or collection?

(A) size()
(B) count()
(C) len()
(D) dimension()

3. How are individual characters accessed in a string s?

(A) s(i)
(B) s[i]
(C) s{i}
(D) s.get(i)

4. What does string slicing s[1:5] extract from string s?

(A) Characters from index 1 to 5 inclusive
(B) Characters from index 1 to 4 inclusive (5 is exclusive)
(C) Characters from index 0 to 4
(D) Every 5th character starting at 1

5. Which string method searches for a substring and returns -1 if not found?

(A) index()
(B) find()
(C) search()
(D) locate()

6. What does the string method index() raise if the substring is absent?

(A) ValueError
(B) KeyError
(C) IndexError
(D) Returns -1

7. Which string method counts the non-overlapping occurrences of a substring?

(A) tally()
(B) frequency()
(C) count()
(D) sum()

8. Which delimiter is used to define a list literal in Python?

(A) Parentheses ( )
(B) Square brackets [ ]
(C) Curly braces { }
(D) Angle brackets < >

9. Are lists in Python mutable or immutable?

(A) Immutable
(B) Mutable
(C) Constant
(D) Hashable

10. Which operator tests whether an item exists inside a list or tuple?

(A) exists
(B) in
(C) contains
(D) has

11. Which list method adds a single element to the end of the list in-place?

(A) push()
(B) insert()
(C) append()
(D) add()

12. Which list method appends all elements of an iterable to the end of the list?

(A) extend()
(B) append()
(C) concat()
(D) push_all()

13. Which list method inserts an element at a specified index position?

(A) insert(index, element)
(B) add(index, element)
(C) place(index, element)
(D) push(index, element)

14. Which statement deletes an element at index i from list L?

(A) L.delete(i)
(B) del L[i]
(C) remove L[i]
(D) drop(L[i])

15. Which list method removes and returns the element at a specified index (default last)?

(A) pop()
(B) remove()
(C) discard()
(D) extract()

16. Which list method removes the first occurrence of a specific value from the list?

(A) pop()
(B) remove()
(C) discard()
(D) del

17. What error occurs if list.remove(x) is called for an element x not present in the list?

(A) IndexError
(B) KeyError
(C) ValueError
(D) TypeError

18. Which delimiter is used to define a tuple literal in Python?

(A) Square brackets [ ]
(B) Parentheses ( ) or comma-separated values
(C) Curly braces { }
(D) Angle brackets < >

19. Are tuples mutable or immutable in Python?

(A) Mutable
(B) Immutable
(C) Partially mutable
(D) Dynamic

20. How is a single-element tuple containing integer 5 properly defined?

(A) t = (5)
(B) t = (5,)
(C) t = [5]
(D) t = tuple(5)

21. What is the type of t = (5)?

(A) tuple
(B) int
(C) list
(D) singleton

22. What technique allows assigning elements of a tuple to multiple variables simultaneously?

(A) Tuple concatenation
(B) Tuple packing
(C) Tuple unpacking / sequence assignment
(D) Tuple indexing

23. What data structure stores unordered key-value pairs with unique keys in Python?

(A) set
(B) tuple
(C) dict (dictionary)
(D) list

24. Which delimiter is used to enclose dictionary key-value literals?

(A) Square brackets [ ]
(B) Curly braces { }
(C) Parentheses ( )
(D) Pipe symbols | |

25. Which method retrieves a dictionary value by key without raising a KeyError if absent?

(A) dict[k]
(B) dict.get(k, default)
(C) dict.fetch(k)
(D) dict.lookup(k)

26. Which dictionary method returns a dynamic view of all keys?

(A) dict.keys()
(B) dict.all_keys()
(C) dict.index()
(D) dict.names()

27. Which dictionary method returns a dynamic view of all values?

(A) dict.values()
(B) dict.elements()
(C) dict.items()
(D) dict.data()

28. Which dictionary method returns pairs of (key, value) as tuple views?

(A) dict.pairs()
(B) dict.items()
(C) dict.tuples()
(D) dict.entries()

29. Can a mutable list be used as a dictionary key in Python?
- **(A)** Yes, any object can be a key
- **(B)** No, keys must be hashable and immutable
- **(C)** Yes, if list length <= 2
- **(D)** Yes, in Python 3.10+

30. Can a tuple containing only strings and integers be used as a dictionary key?
- **(A)** Yes, because it is strictly hashable and immutable
- **(B)** No, tuples are not allowed as keys
- **(C)** Only if length is odd
- **(D)** No, only strings are keys

31. What does the expression ' '.join(['CSE', 'Python', 'AI']) produce?
- **(A)** 'CSE Python AI'
- **(B)** '['CSE', 'Python', 'AI']'
- **(C)** 'CSE, Python, AI'
- **(D)** TypeError

32. What does 'apple,banana,orange'.split(',') produce?
- **(A)** ['apple', 'banana', 'orange']
- **(B)** ('apple', 'banana', 'orange')
- **(C)** {'apple', 'banana', 'orange'}
- **(D)** 'apple banana orange'

33. What does ' hello '.strip() return?
- **(A)** 'hello'
- **(B)** ' hello'
- **(C)** 'hello '
- **(D)** 'h e l l o'

34. What does the list method clear() do?
- **(A)** Deletes the list variable from memory
- **(B)** Removes all elements from the list, making it empty []
- **(C)** Sets all elements to zero
- **(D)** Sorts the list

35. What is the result of [1, 2] + [3, 4] in Python?
- **(A)** [4, 6]
- **(B)** [1, 2, 3, 4]
- **(C)** [[1, 2], [3, 4]]
- **(D)** Error

36. What is the result of [0] * 4 in Python?
- **(A)** [0, 0, 0, 0]
- **(B)** [0, 4]
- **(C)** 0
- **(D)** TypeError

37. Which operator removes a key-value pair from a dictionary by key?
- **(A)** del dict[key]
- **(B)** dict.drop(key)
- **(C)** remove dict[key]
- **(D)** dict - key

38. What does dict.pop(key) return?
- **(A)** The removed key
- **(B)** The value corresponding to the removed key
- **(C)** The remaining dictionary
- **(D)** True

39. What happens when accessing d['unknown'] for a non-existent key using bracket indexing?
- **(A)** Returns None
- **(B)** Raises KeyError
- **(C)** Raises IndexError
- **(D)** Creates empty key

40. What is the boolean truth value of an empty dictionary {}?
- **(A)** True
- **(B)** False
- **(C)** None
- **(D)** Error

41. What is the output of print('Python'[::-1])?
- **(A)** 'nohtyP'
- **(B)** 'Python'
- **(C)** 'P'
- **(D)** IndexError

42. What is the output of:\na = [1, 2, 3]\nb = a\nb.append(4)\nprint(len(a))
- **(A)** 3
- **(B)** 4 (aliasing: both refer to the same list)
- **(C)** None
- **(D)** Error

43. What is the output of:\na = [1, 2, 3]\nb = a[:]\nb.append(4)\nprint(len(a))

(A) 3 (shallow slice copy prevents mutation of a)
(B) 4
(C) 7
(D) Error

44. What is the output of:\nx, y = (10, 20)\nx, y = y, x\nprint(x, y)

(A) 10 20
(B) 20 10
(C) (20, 10)
(D) (10, 20)

45. What does the expression first, *rest = [1, 2, 3, 4] assign to rest?

(A) (2, 3, 4)
(B) [2, 3, 4]
(C) 4
(D) 2

46. What does *head, tail = (10, 20, 30, 40) assign to head?

(A) [10, 20, 30]
(B) (10, 20, 30)
(C) 10
(D) [40]

47. What is the output of print(sorted([3, 1, 4, 1, 5])) vs list.sort()?

(A) sorted() returns a new sorted list; list.sort() sorts in-place returning None
(B) Both return a new list
(C) Both sort in-place
(D) sorted() is for strings only

48. What is the output of print([x**2 for x in range(5) if x % 2 != 0])?

(A) [1, 9]
(B) [0, 1, 4, 9, 16]
(C) [1, 4, 9]
(D) [9, 25]

49. What is the output of:\nd = {'a': 1, 'b': 2}\nprint(d.get('c', 99))

(A) None
(B) KeyError
(C) 99
(D) 0

50. What is the output of:\nd = {'x': 10}\nprint(d.setdefault('x', 50), d.setdefault('y', 50))

(A) 10 50
(B) 50 50
(C) 10 None
(D) KeyError

51. What occurs when attempting to modify an element inside a tuple: t = (1, 2); t[0] = 9?

(A) t becomes (9, 2)
(B) TypeError: 'tuple' object does not support item assignment
(C) ValueError: immutable sequence
(D) IndexError

52. Consider: t = (1, [2, 3]); t[1].append(4). What is the state of t?

(A) (1, [2, 3, 4]) (mutable list inside immutable tuple is mutated)
(B) TypeError is raised
(C) (1, [2, 3])
(D) Tuple is cloned

53. What does dict.fromkeys(['a', 'b'], 0) produce?

(A) {'a': 0, 'b': 0}
(B) {0: 'a', 0: 'b'}
(C) ['a', 'b', 0]
(D) TypeError

54. What is the danger of dict.fromkeys(['a', 'b'], [])?

(A) SyntaxError
(B) Both keys reference the exact same mutable list object in memory
(C) Lists cannot be values
(D) Only one key is created

55. What is the output of:\ns = 'Mississippi'\nprint(s.count('iss'))

(A) 2
(B) 1
(C) 4
(D) 0

56. What is the output of:\ns = 'Engineering'\nprint(s.replace('e', 'X'))

(A) 'EnginXXring'
(B) 'EnginXXring' (case sensitive: only lowercase 'e' replaced)
(C) 'XnginXXring'
(D) 'Engineering'

57. What is the output of:\na = [[0] * 2] * 2\na[0][0] = 5\nprint(a)

(A) [[5, 0], [0, 0]]
(B) [[5, 0], [5, 0]] (outer multiplication replicates row references!)
(C) [[5, 5], [5, 5]]
(D) [[0, 0], [0, 0]]

58. How should an independent 2D matrix of size R x C initialized to 0 be safely created?

(A) [[0] * C] * R
(B) [[0 for _ in range(C)] for _ in range(R)]
(C) [[0] * R * C]
(D) list(2D, 0)

59. What is the output of:\nmatrix = [[1, 2], [3, 4]]\nprint([row[0] for row in matrix])

(A) [1, 2]
(B) [1, 3] (column 0 extraction)
(C) [3, 4]
(D) [2, 4]

60. What does the expression {x: x**3 for x in range(3)} create?

(A) Dictionary comprehension: {0: 0, 1: 1, 2: 8}
(B) Set comprehension: {0, 1, 8}
(C) List: [(0,0), (1,1), (2,8)]
(D) Tuple: ((0,0), ...)

61. What is the output of:\nwords = ['apple', 'pear', 'banana']\nwords.sort(key=len)\nprint(words)

(A) ['pear', 'apple', 'banana']
(B) ['apple', 'banana', 'pear']
(C) ['banana', 'apple', 'pear']
(D) ['pear', 'banana', 'apple']

62. What is the output of:\nd = {'a': 1, 'b': 2}\nfor k in d:\n print(k, end=' ')

(A) a b
(B) 1 2
(C) ('a', 1) ('b', 2)
(D) None

63. What does the dictionary method popitem() do in Python 3.7+?

(A) Removes and returns an arbitrary key-value pair
(B) Removes and returns the last inserted (key, value) pair in LIFO order
(C) Removes the smallest key
(D) Clears the dictionary

64. Since Python 3.7, what guarantee is officially part of Python's dictionary specification?

(A) Keys are automatically sorted numerically
(B) Insertion order of keys is strictly preserved
(C) Dictionaries consume 50% less RAM than lists
(D) Keys are immutable integers

65. What is the output of: list('abc')?

(A) ['a', 'b', 'c']
(B) ['abc']
(C) ('a', 'b', 'c')
(D) ['a b c']

66. What is the output of: tuple('cat')?

(A) ('c', 'a', 't')
(B) ('cat',)
(C) ['c', 'a', 't']
(D) TypeError

67. What is the output of:\nkeys = ['id', 'name']; vals = [101, 'Ada']\nprint(dict(zip(keys, vals)))

(A) {'id': 101, 'name': 'Ada'}
(B) {101: 'id', 'Ada': 'name'}
(C) [('id', 101), ('name', 'Ada')]
(D) TypeError

68. What does the copy.copy() function perform?

(A) Deep recursive copy
(B) Shallow copy (duplicates container, but shares internal child references)
(C) Bitwise clone
(D) Hard link

69. What does copy.deepcopy() perform?

(A) Shallow copy
(B) Deep recursive clone (recursively duplicates all nested container objects)
(C) Memory pointer alias
(D) String serialization

70. What is the output of print(min(['apple', 'Banana', 'cherry'])) based on ASCII values?

(A) 'Banana' (uppercase letters have lower ASCII codes 65-90 than lowercase 97-122)
(B) 'apple'
(C) 'cherry'
(D) TypeError

71. What does 'Python Programming'.find('Pro', 5, 15) return?

(A) 7
(B) -1
(C) 0
(D) 5

72. What is the output of:\nnums = [10, 20, 30, 40]\nnums[1:3] = [99]\nprint(nums)

(A) [10, 99, 40]
(B) [10, 99, 30, 40]
(C) [10, 20, 99, 40]
(D) IndexError

73. What happens when executing nums[1:1] = [5, 6] on list nums = [1, 2]?

(A) Replaces element at index 1
(B) Slices without deletion and inserts [5, 6] at index 1 -> [1, 5, 6, 2]
(C) Raises ValueError
(D) IndexError

74. How is a sparse matrix with zero entries at most coordinates efficiently modeled in Python?

(A) A nested list of all zeros
(B) A dictionary mapping coordinate tuples (row, col) to non-zero values
(C) A tuple of strings
(D) An unindexed linked list

75. What is the output of:\nd = {(0, 1): 5, (2, 3): 8}\nprint(d.get((1, 1), 0))

(A) 5
(B) 8
(C) 0 (default value for zero-entry coordinate)
(D) KeyError

76. What underlying data structure powers Python's list type in CPython?

(A) Singly linked list
(B) Doubly linked list
(C) Dynamic contiguous array of pointers (realloc buffer with overallocation)
(D) B-Tree

77. What is the amortized time complexity of list.append() in CPython?

(A) O(N)
(B) O(1) amortized constant time
(C) O(log N)
(D) O(N^2)

78. What is the worst-case time complexity of list.insert(0, val) at the beginning of a list of size N?

(A) O(1)
(B) O(N) (all N pointers must be shifted in memory)
(C) O(log N)
(D) O(N^2)

79. Which collection in the collections module provides O(1) double-ended append and pop operations?

(A) collections.namedtuple
(B) collections.deque
(C) collections.OrderedDict
(D) collections.ChainMap

80. What collision resolution strategy is employed by CPython's dictionary hash table?

(A) Separate Chaining with linked lists
(B) Open Addressing with pseudo-random perturbation probing
(C) Cuckoo Hashing
(D) Hopscotch Hashing

81. What is the average time complexity of dictionary key lookup d[k] in Python?

(A) O(1) constant time
(B) O(N) linear time
(C) O(log N)
(D) O(k)

82. What is the worst-case time complexity of dictionary key lookup when hash collisions are rampant?

(A) O(1)
(B) O(N)
(C) O(N^2)
(D) O(log N)

83. What built-in function returns the hash code integer of an object for hash table indexing?

(A) id()
(B) hash()
(C) code()
(D) crc32()

84. Why are unhashable types (like list or dict) rejected as dictionary keys?

(A) They lack __hash__() or define __hash__ = None because their hash would change upon mutation
(B) They use too much memory
(C) They are too slow to parse
(D) They conflict with C memory layout

85. What occurs if two distinct objects produce the exact same hash() value in Python (hash collision)?

(A) TypeError is raised immediately
(B) Python uses open addressing probing and resolves equality via __eq__()
(C) The previous value is overwritten silently
(D) The dictionary crashes

86. In Python 3.6+, how did Raymond Hettinger's compact dictionary design reduce memory usage by 20-25%?

(A) By separating the dense sparse hash table array from a compact array of entries [hash, key, val]
(B) By compressing keys using zlib
(C) By storing values on the disk paging file
(D) By eliminating Unicode headers

87. What is the time complexity of the list slice operation L[a:b] where k = b - a?

(A) O(1)
(B) O(k) (copies k pointers into a new PyListObject)
(C) O(N)
(D) O(N log N)

88. What is the time complexity of deleting an element from the middle of a list: del L[i]?

(A) O(1)
(B) O(N - i) (requires shifting remaining pointers leftward)
(C) O(N log N)
(D) O(log N)

89. What is the sorting algorithm used by Python's list.sort() and sorted()?

(A) QuickSort
(B) HeapSort
(C) Timsort (hybrid of Merge Sort and Insertion Sort)
(D) RadixSort

90. What is the worst-case time complexity and stability of Timsort?

(A) O(N^2), Unstable
(B) O(N log N), Stable (preserves relative order of equal elements)
(C) O(N), Stable
(D) O(N log N), Unstable

91. What is the best-case time complexity of Timsort on an already sorted or mostly sorted array?

(A) O(N) linear time
(B) O(N log N)
(C) O(1)
(D) O(N^2)

92. What does collections.defaultdict(list) do when an accessed key does not exist?

(A) Raises KeyError
(B) Automatically calls list() to instantiate an empty list [] as value and inserts key
(C) Returns None
(D) Raises TypeError

93. What does collections.Counter('banana') produce?

(A) Counter({'a': 3, 'n': 2, 'b': 1})
(B) {'b': 1, 'a': 1, 'n': 1}
(C) 6
(D) ['b', 'a', 'n']

94. What is the time complexity of string concatenation in a loop: s = ''; for c in words: s += c?

(A) O(N) linear time
(B) O(N^2) quadratic time due to repeated string reallocations and memory copies
(C) O(1)
(D) O(log N)

95. Why is ''.join(words) dramatically faster than repeated += in a loop?

(A) join() calculates total buffer length in advance and allocates memory once in O(N) time
(B) join() uses multi-threading
(C) join() skips whitespace
(D) join() compresses strings

96. What is the output of:\na = [[]] * 3\na[0].append('X')\nprint(a)

(A) [['X'], [], []]
(B) [['X'], ['X'], ['X']] (all 3 elements reference the identical inner list)
(C) ['X', 'X', 'X']
(D) TypeError

97. What is the result of hash(float('nan')) in Python?

(A) Zero always
(B) A valid integer hash, but nan != nan causes unexpected hash table behavior
(C) TypeError: unhashable float
(D) Raises FloatingPointError

98. What happens if float('nan') is inserted into a set: s = {float('nan'), float('nan')}?

(A) Set contains 1 element
(B) Set contains 2 elements because nan != nan, bypassing equality checks
(C) TypeError
(D) ValueError

99. What is the space complexity of tuple unpacking: a, b, c = t?

(A) O(len(t))
(B) O(1) pointer register assignment
(C) O(N^2)
(D) O(log N)

100. What does the dictionary union operator d1 | d2 (introduced in Python 3.9) perform?

(A) Bitwise OR of keys
(B) Creates a merged dictionary combining keys/values of d1 and d2 (d2 overwriting d1)
(C) Intersection of keys
(D) TypeError

SECTION B: DESCRIPTIVE & SUBJECTIVE QUESTIONS

Q1. Explain Strings as Compound Data Types in Python. Detail string traversal algorithms using for and while loops, character length determination, and memory immutability.

Q2. Explain String Slicing, Negative Indexing, and Lexicographical String Comparison in Python with comprehensive code examples.

Q3. Discuss String Searching and Transformation Methods: find(), index(), count(), replace(), split(), join(), and strip(). Explain exceptions vs sentinel returns.

Q4. Explain Python Lists: Dynamic Array Architecture, Memory Allocation, Mutability, and In-Place Mutation versus Rebinding.

Q5. Explain List Slicing, Slice Assignment, Deletion (del, remove, pop, clear), and Element Access with Negative Indexing.

Q6. Explain List Comprehensions in Python. Contrast list comprehensions with map(), filter(), and explicit for loops in readability, bytecode, and performance.

Q7. Explain Parameter Passing in Python: 'Pass-by-Object-Reference' (Call-by-Sharing). Demonstrate how mutating mutable parameters affects caller state.

Q8. Explain Tuples in Python: Immutability, Creation, Single-Element Gotcha, Tuple Packing, Unpacking, and Functions Returning Multiple Values.

Q9. Explain Python Dictionaries: Hash Table Architecture, Key Hashability, Collision Resolution, and Compact Dict Memory Layout.

Q10. Discuss Dictionary Methods: get(), keys(), values(), items(), update(), setdefault(), and Dictionary Comprehensions with practical applications.

Q11. Explain Sparse Matrices. Compare traditional 2D nested list representation with Dictionary of Keys (DOK) in memory footprint and retrieval efficiency.

Q12. Explain Aliasing, Shallow Copy, and Deep Copy in Python. Analyze the copy module (copy() vs deepcopy()) with memory address diagrams and nested mutation traces.

Q13. Implement a Text Analytics & Word Frequency Counter in Python using strings, lists, dictionaries, and collections.Counter with punctuation stripping.

Q14. Implement a Student Gradebook Management System in Python using nested dictionaries, calculating per-student weighted GPAs and class statistics.

Q15. Implement Matrix Multiplication in Python using nested lists and nested loops, validating inner matrix dimension compatibility.

Q16. Explain the collections.namedtuple factory in Python. Compare namedtuple with standard tuples, dictionaries, and classes in syntax and memory overhead.

Q17. Implement a Two-Sum Algorithm in Python using Hash Maps (Dictionaries) to achieve O(N) linear time complexity versus O(N^2) brute force.

Q18. Implement an LRU (Least Recently Used) Cache simulation in Python using OrderedDict or dictionaries with move_to_end().

Q19. Design an Anagram Grouping program in Python using dictionaries, sorted string keys, or character frequency tuples.

Q20. Implement a Stack and Queue Data Structure in Python using list vs collections.deque. Benchmark pop(0) vs popleft() performance.

Q21. Implement the Sieve of Eratosthenes in Python using boolean lists and list indexing to compute all prime numbers up to N efficiently.

Q22. Implement a JSON-Style Nested Data Parser and Flattener in Python using dictionaries, recursive traversal, and key concatenation.

Q23. Implement a Longest Common Prefix (LCP) algorithm in Python across an array of strings using character-by-character vertical scanning and zip().

Q24. Explain the collections.defaultdict type. Contrast standard dict.setdefault() with defaultdict and demonstrate grouping applications.

Q25. Implement a Sudoku Board Validator in Python using 2D nested lists, sets, and coordinate slicing to verify rows, columns, and 3x3 subgrids.

Q26. Implement a String Permutation and Combination generator in Python without external libraries using recursive backtracking and list swapping.

Q27. Explain the collections.ChainMap class in Python. How does it search across multiple chained dictionaries and how do updates behave?

Q28. Implement an In-Memory Inverted Index for a Document Search Engine using Python dictionaries and sets.

Q29. Design an Inventory Management System in Python using dictionaries, handling stock reorders, low-stock alerts, and batch transactions.

Q30. Implement a Custom Matrix Determinant and Matrix Inversion program for 2x2 and 3x3 matrices in Python using nested lists.

SECTION C: ANSWER KEY - MULTIPLE CHOICE QUESTIONS

Q1:	[D]	**Q21:**	[B]	**Q41:**	[A]	**Q61:**	[A]	**Q81:**	[A]
Q2:	[C]	**Q22:**	[C]	**Q42:**	[B]	**Q62:**	[A]	**Q82:**	[B]
Q3:	[B]	**Q23:**	[C]	**Q43:**	[A]	**Q63:**	[B]	**Q83:**	[B]
Q4:	[B]	**Q24:**	[B]	**Q44:**	[B]	**Q64:**	[B]	**Q84:**	[A]
Q5:	[B]	**Q25:**	[B]	**Q45:**	[B]	**Q65:**	[A]	**Q85:**	[B]
Q6:	[A]	**Q26:**	[A]	**Q46:**	[A]	**Q66:**	[A]	**Q86:**	[A]
Q7:	[C]	**Q27:**	[A]	**Q47:**	[A]	**Q67:**	[A]	**Q87:**	[B]
Q8:	[B]	**Q28:**	[B]	**Q48:**	[A]	**Q68:**	[B]	**Q88:**	[B]
Q9:	[B]	**Q29:**	[B]	**Q49:**	[C]	**Q69:**	[B]	**Q89:**	[C]
Q10:	[B]	**Q30:**	[A]	**Q50:**	[A]	**Q70:**	[A]	**Q90:**	[B]
Q11:	[C]	**Q31:**	[A]	**Q51:**	[B]	**Q71:**	[A]	**Q91:**	[A]
Q12:	[A]	**Q32:**	[A]	**Q52:**	[A]	**Q72:**	[A]	**Q92:**	[B]
Q13:	[A]	**Q33:**	[A]	**Q53:**	[A]	**Q73:**	[B]	**Q93:**	[A]
Q14:	[B]	**Q34:**	[B]	**Q54:**	[B]	**Q74:**	[B]	**Q94:**	[B]
Q15:	[A]	**Q35:**	[B]	**Q55:**	[A]	**Q75:**	[C]	**Q95:**	[A]
Q16:	[B]	**Q36:**	[A]	**Q56:**	[B]	**Q76:**	[C]	**Q96:**	[B]
Q17:	[C]	**Q37:**	[A]	**Q57:**	[B]	**Q77:**	[B]	**Q97:**	[B]
Q18:	[B]	**Q38:**	[B]	**Q58:**	[B]	**Q78:**	[B]	**Q98:**	[B]
Q19:	[B]	**Q39:**	[B]	**Q59:**	[B]	**Q79:**	[B]	**Q99:**	[B]
Q20:	[B]	**Q40:**	[B]	**Q60:**	[A]	**Q80:**	[B]	**Q100:**	[B]

MCQ Scoring Guide & Difficulty Distribution:

Questions 1 to 40: Foundational & Conceptual [Easy] | Questions 41 to 75: Analytical & Applied [Moderate]

Questions 76 to 100: Complex Numerical & Advanced Problem Solving [Difficult]

SECTION B: DETAILED SOLUTIONS FOR SUBJECTIVE QUESTIONS

```
sample_str = "CSE-Python 3.10 & DataStructures!"
analysis = traverse_and_analyze_string(sample_str)
print("String Traversal & Analysis Report:", analysis)
```

4. Memory Model & Execution Trace:
- Stack: text -> Heap [PyUnicodeObject: length=33, state=compact_ascii, buffer='CSE-Python...']
- Each char during iteration is a transient reference to interned single-character string singletons in CPython memory.
- No in-place modification occurs; total allocated buffer remains constant.

5. Complexity Analysis:
- Time Complexity: O(N) linear time single pass over N characters.
- Space Complexity: O(1) auxiliary counter storage.

>>Solution to Question 2: Explain String Slicing, Negative Indexing, and Lexicographical String Comparison in Python with comprehensive code examples.

```
print(f"'{s1}' < '{s2}': {s1 < s2} (Reason: 'A'=65 < 'a'=97)")
print(f"'Data' == 'data'.capitalize(): {'Data' == 'data'.capitalize()}")
print(f"'byte' < 'bytes': {'byte' < 'bytes'} (Prefix extension)")

demonstrate_slicing_and_comparisons()
```

5. Complexity Analysis:
- Slicing Time Complexity: O(K) where K is slice length (allocates new PyUnicodeObject).
- Lexicographical Comparison: O(min(len(s1), len(s2))) character comparisons.

>>P Solution Statement Question 3: Discuss String Searching and Transformation Methods: find(), index(), count(), replace(), split(), join(), and strip(). Explain exceptions vs

- find() / count() Time: O(N) using Boyer-Moore-Horspool / two-way algorithm.
- split() / join() Time: O(N) memory allocation and copy.
- Space Complexity: O(N) memory for generated lists and strings.

Solution to Question 4: Explain Python Lists: Dynamic Array Architecture, Memory Allocation, Mutability, and In-Place Mutation versus Rebinding.

```
print(f"Rebinding via + -> ID Changed: {id(lst1) != orig_id}")

5. Complexity Analysis:
- append() / pop(): Amortized O(1) time.
- insert(0, x) / pop(0): O(N) linear time (shifts N pointers).
- Memory Overhead: O(N) contiguous pointer buffer.
```

Solution to Question 5: Explain List Slicing, Slice Assignment, Deletion (del, remove, pop, clear), and Element Access with Negative Indexing.

```
>>> # Solution to Question 5:
```

- Indexing: O(1) time.
- Slice Assignment: O(N + K) where K is number of inserted elements.
- Space Complexity: In-place pointer modifications.

>>> Solution to Question 6: Explain List Comprehensions in Python. Contrast list comprehensions with map(), filter(), and explicit for loops in readability, bytecode,

- Time Complexity: O(N) linear time evaluation.
- Space Complexity: O(N) memory allocated for the new list.

Solution to Question 57: Main Parameter Passing in Python: 'Pass-by-Object-Reference' (Call-by-Sharing), Demonstrate how mutating mutable parameters a

---

Solution Statement Question 8: Explain Tuples in Python: Immutability, Creation, Single-Element Gotcha, Tuple Packing, Unpacking, and Functions Returning Multiple

---

Solution to Question 9: Explain Python Dictionaries: Hash Table Architecture, Key Hashability, Collision Resolution, and Compact Dict Memory Layout.

```
>>> # Solution to Question 9:
```

6. Complexity Analysis:
- Average Lookup / Insert / Delete Time: O(1) constant time.
- Worst-Case Time (Massive Collisions): O(N).
- Space Complexity: O(N) compact dense array.

Solution to Question 10:

Solution to Question 11: >>> Explain Sparse Matrices. Compare traditional 2D nested list representation with Dictionary of Keys (DOK) in memory footprint and retri

- Retrieval Time: O(1) average hash lookup.

Solution to Question 12: Explain Aliasing, Shallow Copy, and Deep Copy in Python. Analyze the copy module (copy() vs deepcopy()) with memory address diag

---

Solution Statement: Implement a Text Analytics & Word Frequency Counter in Python using strings, lists, dictionaries, and collections.Counter with punctua

- Frequency Counting: O(W) where W is word count.
- Top K Extraction: O(W log K) using min-heap inside Counter.most_common().
- Space Complexity: O(W) memory for token list and frequency map.

Solution to Question 14:

```
>>> Implement a Student Gradebook Management System in Python using nested dictionaries, calculating per-student weighted GPAs and
```

```
gb.add_student("CS101", "Aarav Sharma")
gb.add_course_grade("CS101", "CS101", credits=4, grade="A")
gb.add_course_grade("CS101", "MA101", credits=4, grade="B")

gb.add_student("CS102", "Priya Patel")
gb.add_course_grade("CS102", "CS101", credits=4, grade="A")
gb.add_course_grade("CS102", "MA101", credits=4, grade="A")

gb.display_report()
```

4. Complexity Analysis:
- Time Complexity: O(1) student/course insertion; O(C) GPA calculation per student.
- Space Complexity: O(S * C) nested dictionary memory.

Solution to Question 15:

```
>>> Implement Matrix Multiplication in Python using nested lists and nested loops, validating inner matrix dimension compatibility.
```

```
matB = [[7, 8],
[9, 1],
[2, 3]] # 3 x 2

product = multiply_matrices_checked(matA, matB)
print(f"Matrix A (2x3) x Matrix B (3x2) = Product (2x2):")
for r in product:
print(" ", r)
```

3. Complexity Analysis:
- Time Complexity: O(M * K * N) arithmetic multiplications and additions.
- Space Complexity: O(M * N) memory for product matrix C.

>>PSolution to Question 16 Explain the collections.namedtuple factory in Python. Compare namedtuple with standard tuples, dictionaries, and classes in syntax an

```
raw_tuple = (10.5, 20.0, -5.2)
raw_dict = {'x': 10.5, 'y': 20.0, 'z': -5.2}

print(f"\nMemory Size of namedtuple: {sys.getsizeof(p1)} bytes")
print(f"Memory Size of raw tuple: {sys.getsizeof(raw_tuple)} bytes (Exact same!)")
print(f"Memory Size of dictionary: {sys.getsizeof(raw_dict)} bytes (5x larger!)")

4. Complexity Analysis:
- Field Access Time: O(1) direct struct offset lookup.
- Space Complexity: O(1) strictly minimal tuple allocation.
```

>>P Solution to Question 17: Implement a Two-Sum Algorithm in Python using Hash Maps (Dictionaries) to achieve O(N) linear time complexity versus O(N^2) brute

- Iteration 0: num=2, comp=7. seen={2: 0}
- Iteration 1: num=7, comp=2. 2 is found in seen! Return (seen[2], 1) = (0, 1).

5. Complexity Analysis:
- Time Complexity: O(N) single linear pass with O(1) hash table lookups.
- Space Complexity: O(N) auxiliary space to store seen hash map.

Solution to Question 18:

>>> Implement an LRU (Least Recently Used) Cache simulation in Python using OrderedDict or dictionaries with move_to_end().

```
lru.display() # Key 1 is now most recent; Key 2 is oldest

lru.put(3, 30) # Evicts Key 2!
lru.display()
```

3. Complexity Analysis:
- Time Complexity: O(1) for both get() and put() operations.
- Space Complexity: O(capacity) memory bound.

>> Solution to Question 20: Write an Anagram Grouping program in Python using dictionaries, sorted string keys, or character frequency tuples.

[Problem Statement]: Implement a Stack and Queue Data Structure in Python using list vs collections.deque. Benchmark pop(0) vs popleft() perf
[Detailed Step-by-Step Answer]:
1. Data Structure Mechanics:
- Stack: Last-In-First-Out (LIFO). Supported natively by list via append() (push) and pop()
(pop) in O(1) time.
- Queue: First-In-First-Out (FIFO). Requires enqueue at one end and dequeue at opposite end.

2. The Python List Pop(0) Performance Trap:
In Python list, pop(0) removes the first pointer, forcing CPython to shift all remaining N - 1
pointers in memory leftward via memmove(), incurring O(N) linear time overhead.
Using a list as a queue leads to disastrous O(N^2) cumulative performance!
collections.deque (Double-Ended Queue) is implemented as a doubly linked list of fixed-size
memory blocks (64 elements each). Both append() and popleft() run in strictly O(1) time without
pointer shifting!

3. Python Code Demonstration & Micro-Benchmark:

```
import time
from collections import deque

def benchmark_queue_performance(n=50_000):
# Benchmark 1: list.pop(0) - O(N)
lst_q = list(range(n))
start_t = time.perf_counter()
while lst_q:
lst_q.pop(0)
list_elapsed = time.perf_counter() - start_t

# Benchmark 2: deque.popleft() - O(1)
deq_q = deque(range(n))
start_t = time.perf_counter()
while deq_q:
deq_q.popleft()
deque_elapsed = time.perf_counter() - start_t

print(f"Benchmark: Dequeuing {n:,} items:")
print(f" list.pop(0) [O(N)] Time: {list_elapsed:.4f} seconds")
print(f" deque.popleft() [O(1)] Time: {deque_elapsed:.6f} seconds")
print(f" Speedup Factor: {list_elapsed / deque_elapsed:.1f}x faster!")

benchmark_queue_performance(30_000)
```

4. Complexity Analysis:
- list.pop(0) Time Complexity: O(N).
- deque.popleft() Time Complexity: O(1).
- Space Complexity: O(N) memory allocation.

>> Solution to Question 21: [Problem Statement]: Implement the Sieve of Eratosthenes in Python using boolean lists and list indexing to compute all prime numbers up to N efficiently.

False.
2. For p = 2 up to floor(sqrt(N)):
If is_prime[p] is True, mark all multiples of p starting from p^2 up to N as False:
for i in range(p * p, N + 1, p): is_prime[i] = False.
3. Collect all indices where is_prime[i] is True.

2. Python Implementation:

```
def generate_primes_sieve(limit: int) -> list:
"""Compute primes up to limit using Sieve of Eratosthenes."""
if limit < 2:
return []

is_prime = [True] * (limit + 1)
is_prime[0] = is_prime[1] = False

# Check up to sqrt(limit)
for p in range(2, int(limit ** 0.5) + 1):
if is_prime[p]:
# Slice assignment or strided range loop
for multiple in range(p * p, limit + 1, p):
is_prime[multiple] = False

# List comprehension to collect primes
primes = [num for num, prime_flag in enumerate(is_prime) if prime_flag]
return primes

# Verification
prime_list = generate_primes_sieve(100)
print(f"Primes up to 100 ({len(prime_list)} primes found):")
print(prime_list)
```

3. Complexity Analysis:
- Time Complexity: O(N log(log N)) harmonic prime series convergence.
- Space Complexity: O(N) boolean flags in memory.

>> Solution to Question 22: Implement a JSON-Style Nested Data Parser and Flattener in Python using dictionaries, recursive traversal, and key concatenation.

```
# Recursively flatten inner dictionary
items.extend(flatten_dictionary(v, new_key, sep=sep).items())
elif isinstance(v, list):
# Flatten lists with index annotations
for idx, elem in enumerate(v):
list_key = f"{new_key}[{idx}]"
if isinstance(elem, dict):
items.extend(flatten_dictionary(elem, list_key, sep=sep).items())
else:
items.append((list_key, elem))
else:
items.append((new_key, v))

return dict(items)

# Verification
raw_json = {
"server": "prod-east-1",
"database": {
"host": "10.0.0.1",
"port": 5432,
"credentials": {
"username": "db_admin",
"ssl_enabled": True
}
},
"tags": ["web", "finance"]
}

flat = flatten_dictionary(raw_json)
print("Flattened Dictionary Keys & Values:")
for k, v in flat.items():
print(f" {k:<35} -> {v}")
```

3. Complexity Analysis:
- Time Complexity: O(Total Key-Value Nodes) single traversal.
- Space Complexity: O(Depth) call stack + O(Nodes) flattened dictionary.

>> Solution to Question 23: Implement a Longest Common Prefix (LCP) algorithm in Python across an array of strings using character-by-character vertical scanning

```
if len(strs) == 1:
return strs[0]

# Vertical scanning via zip(*strs)
prefix = []
for char_tuple in zip(*strs):
# If all characters in the column are identical
if len(set(char_tuple)) == 1:
prefix.append(char_tuple[0])
else:
break

return "".join(prefix)

# Verification
test1 = ["interspecies", "interstellar", "interstate"]
test2 = ["throne", "dungeon"]
test3 = ["prefix", "preheat", "preview"]

print(f"LCP of {test1} -> '{longest_common_prefix_vertical(test1)}'")
print(f"LCP of {test2} -> '{longest_common_prefix_vertical(test2)}'")
print(f"LCP of {test3} -> '{longest_common_prefix_vertical(test3)}'")
```

3. Complexity Analysis:
- Time Complexity: O(S) where S is sum of characters across all strings (best case O(M * min_len)).
- Space Complexity: O(min_len) memory for prefix buffer.

>> Solution to Question 24: Explain the collections.defaultdict type. Contrast standard dict.setdefault() with defaultdict and demonstrate grouping applications.

```
("Marketing", "Rohan")
]

dept_map = defaultdict(list)
for dept, employee in records:
dept_map[dept].append(employee) # Clean, no KeyError checks needed!

print("Department Roster via defaultdict(list):")
for dept, roster in dept_map.items():
print(f" {dept:<15}: {roster}")

# 2. Integer Accumulator via defaultdict(int)
inventory = [("apple", 10), ("banana", 5), ("apple", 15), ("orange", 8)]
stock = defaultdict(int)
for item, qty in inventory:
stock[item] += qty # Initializes missing key with int() = 0

print(f"\nAggregated Inventory: {dict(stock)}")

3. Complexity Analysis:
- Lookup and Default Insertion: O(1) amortized time.
- Space Complexity: O(K) where K is number of unique keys.
```

>> Solution to Question 25: Implement a Sudoku Board Validator in Python using 2D nested lists, sets, and coordinate slicing to verify rows, columns, and 3x3 subg

```
    return False

    rows[r].add(val)
    cols[c].add(val)
    boxes[box_idx].add(val)

    return True

    # Verification
    sample_board = [
    ["5","3",".",".","7",".",".",".",".","."],
    ["6",".",".","1","9","5",".",".",".","."],
    [".","9","8",".",".",".",".",".","6","."],
    ["8",".",".",".",".","6",".",".",".","3"],
    ["4",".",".","8",".","3",".",".","1"],
    ["7",".",".",".",".","2",".",".",".","6"],
    [".","6",".",".",".",".",".","2","8","."],
    [".",".",".",".","4","1","9",".",".","5"],
    [".",".",".",".",".","8",".",".","7","9"]
    ]

    is_valid = SudokuValidator.is_valid_sudoku(sample_board)
    print(f"Is Sample Sudoku Board Valid? -> {is_valid}")

    3. Complexity Analysis:
    - Time Complexity: O(1) constant time (exactly 81 cells checked).
    - Space Complexity: O(1) constant storage (sets of size <= 9).
```

>> Solution to Question 26: Implement a String Permutation and Combination generator in Python without external libraries using recursive backtracking and list sw

```
    # Backtrack: swap back to restore original order
    chars[index], chars[i] = chars[i], chars[index]

    return results

    # Verification
    input_str = "ABC"
    perms = generate_permutations(list(input_str))
    print(f"Permutations of '{input_str}' (Total: {len(perms)}): {perms}")
```

3. Complexity Analysis:
- Time Complexity: O(N * N!) recursive backtracking.
- Space Complexity: O(N) recursion stack depth.

Solution to Question 27:

>>> Explain the collections.ChainMap class in Python. How does it search across multiple chained dictionaries and how do updates behave

4. Complexity Analysis:
- Time Complexity: O(M) lookup where M is number of chained dictionaries; O(1) writes.
- Space Complexity: O(1) zero data duplication.

Solution to Question 28:

```
>>> # Solution to Question 28: Implement an In-Memory Inverted Index for a Document Search Engine using Python dictionaries and sets.
```

- Query Time: O(min(len(set1), len(set2))) set intersection.
- Space Complexity: O(Unique Terms * Posting Lists).

Solution to Question 29:

>>> Implementing an Inventory Management System in Python using dictionaries, handling stock reorders, low-stock alerts, and batch transaction

3. Complexity Analysis:
- Lookup & Sale Time: O(1) dictionary key indexing.
- Reorder Scan Time: O(N) where N is number of catalog SKUs.
- Space Complexity: O(N) memory for catalog.

Solution to Question 30:

>>> Implement a Custom Matrix Determinant and Matrix Inversion program for 2x2 and 3x3 matrices in Python using nested lists.

3. Complexity Analysis:
- Time Complexity: O(1) closed-form formula evaluations.
- Space Complexity: O(1) 2x2 / 3x3 return matrices.
